Guía de estudio del backend

Esta guía explica el backend de Buena Energía paso a paso, con ejemplos de Laravel y PHP pensados para quien recién empieza. Sigue el recorrido de una solicitud HTTP: desde la ruta, pasando por el controlador y los modelos, hasta la base de datos y la respuesta JSON.

> **Alcance:** el backend es la autoridad para validar datos, precios, existencias y permisos. El frontend muestra la tienda y consume esta API, pero una comprobación hecha solo en el navegador no protege los datos. Las referencias a líneas corresponden a la versión revisada y pueden cambiar a medida que evolucione el código.

1. Qué hace el backend

El backend es la aplicación Laravel que recibe solicitudes del frontend y decide qué operaciones son válidas. En esta tienda se ocupa de:

- Consultar productos y variantes para el catálogo.
- Validar y registrar pedidos con los precios vigentes del servidor.
- Crear desafíos CAPTCHA de corta duración.
- Autenticar administradores y proteger las operaciones de gestión.
- Crear y editar productos, imágenes y variantes.
- Actualizar existencias y estados de pedidos.

La respuesta suele ser JSON. Por ejemplo, el frontend pide `GET /api/productos`; Laravel busca en la base de datos, transforma los modelos a una estructura preparada para el catálogo y devuelve esa estructura.

2. Mapa del proyecto

```
routes/api.php
  └─ decide qué controlador recibe cada URL
      ├─ ProductoController.php
      ├─ PedidoController.php
      ├─ CaptchaController.php
      ├─ AdminAuthController.php
      └─ AdminProductoController.php
          └─ Models/ → Eloquent → base de datos

database/migrations/ → define la estructura de las tablas
database/seeders/    → agrega datos iniciales de desarrollo
tests/Feature/       → prueba rutas y comportamiento con Laravel
bootstrap/app.php    → conecta rutas, middleware y respuestas de error
config/              → configura base de datos, autenticación, caché y CORS
```

- `routes/api.php` declara los endpoints y qué middleware protege las rutas administrativas.
- `PedidoController` recibe solicitudes, valida datos y coordina las operaciones.
- `Producto` representa registros y relaciones de la base de datos con Eloquent.
- `create_productos_table` conserva la historia de cambios del esquema.
- `DatabaseSeeder` prepara productos y, si se configuran credenciales, un administrador inicial.
- `OrderManagementTest` comprueba rutas completas y resultados observables.

- `bootstrap/app.php` conecta rutas y middleware; hace que las rutas `api/*` respondan errores en formato JSON.
- `config/cors.php` define qué orígenes web pueden llamar a la API desde un navegador.

3. Cómo viaja una solicitud

Una solicitud sigue, normalmente, este camino:

1. El navegador envía método, URL, encabezados y, a veces, un cuerpo JSON o un archivo.
2. Laravel compara método y URL con las rutas declaradas.
3. Middleware como `auth:sanctum` verifica identidad y permisos si la ruta es privada.
4. El controlador valida la entrada y aplica las reglas del negocio.
5. Eloquent consulta o modifica la base de datos.
6. Laravel devuelve un código HTTP y un cuerpo JSON.

```
PATCH /api/admin/variantes/42/stock
Authorization: (token bearer de ejemplo omitido)
{"stock": 0}
          │
          ▼
routes/api.php → auth:sanctum → abilities:admin
          │
          ▼
AdminProductoController::updateStock()
          │ valida que stock sea entero y no negativo
          ▼
ProductoVariante → base de datos
          │
          ▼
200 OK + JSON con el stock guardado
```

El ejemplo muestra el recorrido conceptual. El número y el token son ilustrativos; no son credenciales reales.

4. Rutas: la puerta de entrada

En `routes/api.php` cada ruta asocia un verbo HTTP, una URL y una acción del controlador:

```
Route::get('/productos', [ProductoController::class, 'index']);
Route::post('/pedidos', [PedidoController::class, 'store'])
    ->middleware('throttle:5,1');

Route::prefix('admin')->group(function () {
    Route::post('/login', [AdminAuthController::class, 'login']);

    Route::middleware(['auth:sanctum', 'abilities:admin'])->group(function () {
        Route::patch('/variantes/{variantId}/stock', [AdminProductoController::class, 'updateStock']);
    });
});
```

- `GET` suele leer datos; `POST` crea o inicia una operación; `PATCH` modifica una parte de un recurso.
- `prefix('admin')` agrega `/admin` a las rutas de ese grupo.
- `{variantId}` es un valor de la URL que Laravel entrega al controlador.
- `throttle:5,1` limita la frecuencia: en este caso, hasta cinco intentos por minuto para la combinación de ruta y cliente que mida el limitador.
- Las rutas dentro del segundo grupo requieren un token válido y la capacidad `admin`.

Los endpoints declarados actualmente son:

Método	Ruta	Acceso	Para qué sirve
`GET`	`/api/productos`	Público	Listar el catálogo; acepta filtro opcional `categoria`.
`GET`	`/api/captcha/challenge`	Público, con límite	Crear una pregunta de verificación.
`POST`	`/api/captcha/verify`	Público, con límite	Verificar un desafío una sola vez.
`POST`	`/api/pedidos`	Público, con límite	Validar y registrar un pedido pendiente.
`POST`	`/api/admin/register`	Público, con límite y clave	Crear el primer administrador.
`POST`	`/api/admin/login`	Público, con límite	Autenticar un administrador y entregar token.
`POST`	`/api/admin/logout`	Administrador	Revocar el token actual.
`POST`	`/api/admin/productos`	Administrador	Crear producto y variantes.
`PATCH`	`/api/admin/productos/{producto}`	Administrador	Editar un producto y sus variantes.
`PATCH`	`/api/admin/variantes/{variantId}/stock`	Administrador	Cambiar existencias de una variante.
`GET`	`/api/admin/pedidos`	Administrador	Listar pedidos.
`PATCH`	`/api/admin/pedidos/{orderId}`	Administrador	Cambiar estado del pedido.

Para pedirle a Laravel su lista efectiva, usa:

```
php artisan route:list --path=api
```

Consultar la lista de rutas ayuda a distinguir lo que realmente está habilitado de rutas antiguas mencionadas en documentación o ejemplos.

5. Conceptos básicos de PHP y Laravel en este backend

Una clase y un método

Un controlador agrupa acciones relacionadas. Este método pertenece a la clase `ProductoController`:

```
class ProductoController extends Controller
{
    public function index(Request $request)
    {
        // Leer filtros y preparar la respuesta del catálogo.
    }
}
```

`Request` representa la petición entrante. `$request` es el nombre de la variable que permite leer sus parámetros y cuerpo. `public function` declara un método que el enrutador puede invocar.

Arrays asociativos

En PHP, el formato `['nombre' => 'valor']` es un array con claves. Laravel lo usa para validar datos, crear modelos y componer JSON:

```
$validated = $request->validate([
    'nombre' => ['required', 'string', 'max:255'],
]);
```

Si la entrada no cumple las reglas, Laravel interrumpe la acción y devuelve errores de validación. En peticiones API, el cliente recibe normalmente `422` con los campos inválidos.

Eloquent y consultas

Eloquent es el ORM de Laravel: permite trabajar con objetos PHP que representan filas, usando relaciones en vez de escribir todo el SQL a mano.

```
$product = Producto::query()->findOrFail($productId);
$variants = $product->variantes;
```

`findOrFail` busca por clave primaria y produce un `404` si no existe. `$product->variantes` accede a una relación definida por el modelo.

Colecciones

Una colección es una lista de resultados de Eloquent con métodos útiles, como `map`, `filter` y `keyBy`. Por ejemplo, `map` transforma cada registro del catálogo y `keyBy('id_variante')` permite recuperar una variante rápidamente por su identificador.

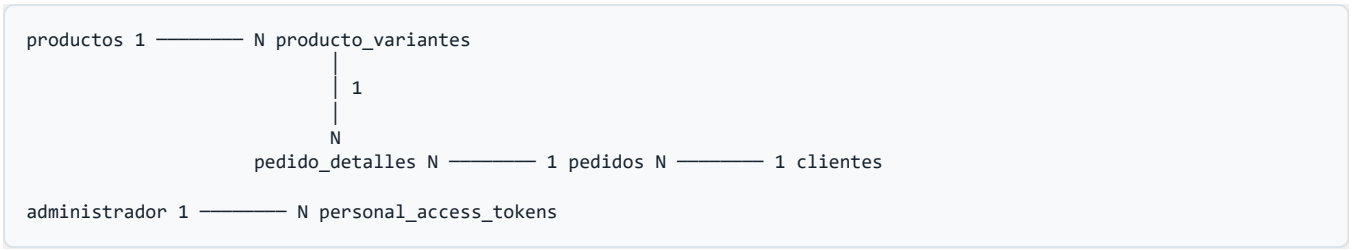
Respuestas HTTP

El código de estado explica el resultado a la otra aplicación:

- `200`: operación correcta que devuelve datos.
- `201`: se creó un recurso, como un pedido o producto.
- `204`: operación correcta sin cuerpo, como cerrar sesión.
- `401`: falta autenticación válida.
- `403`: el usuario está identificado pero no tiene autorización, o una clave fue rechazada.
- `404`: no existe la ruta o el recurso solicitado.
- `422`: la entrada no pasó validación o una regla de negocio.
- `429`: se superó el límite de solicitudes.
- `500`: ocurrió un fallo interno.

6. Datos y relaciones

Las migraciones crean las tablas del dominio. Las tablas principales son:



La lectura del dibujo es: un producto tiene muchas variantes; un detalle de pedido refiere a una variante; un pedido tiene un cliente y muchos detalles. Sanctum guarda los tokens asociados a su administrador en la tabla de tokens.

Tabla	Datos importantes
`productos`	Nombre, descripción, precio, categoría e imagen general.
`producto_variantes`	Producto, talle, color, stock e imagen opcional.
`clientes`	Nombre, teléfono y dirección usados al comprar.
`pedidos`	Cliente, fecha, entrega, estado, total y notas.
`pedido_detalles`	Variante, cantidad y precio unitario histórico.
`administrador`	Nombre, correo único y contraseña con hash.
`personal_access_tokens`	Tokens de Sanctum, permisos y caducidad.

Las migraciones detallan el esquema, incluyendo claves foráneas y tipos: `productos`, `producto_variantes`, `pedidos` y `pedido_detalles`. Una migración posterior añade las notas opcionales del pedido (`add_notas_to_pedidos`).

Modelos y relaciones

Las tablas usan nombres en español y algunas claves primarias no siguen las convenciones predeterminadas de Laravel. Por eso los modelos declaran explícitamente tabla y clave:

```
class Producto extends Model
{
    protected $table = 'productos';
    protected $primaryKey = 'id_producto';

    protected $fillable = ['nombre', 'descripcion', 'precio', 'categoria', 'imagen'];

    public function variantes()
    {
        return $this->hasMany(ProductoVariante::class, 'id_producto', 'id_producto');
    }
}
```

- `protected $table` indica el nombre real de la tabla.
- `$primaryKey` indica la columna identificadora.
- `$fillable` es una lista permitida para asignación masiva con `create()` o `update()`. Reduce el riesgo de guardar campos que no debían venir de la petición.
- `hasMany` expresa que un producto puede tener varias variantes.

La relación inversa usa `belongsTo`: cada variante pertenece a un producto. `Pedido` pertenece a un cliente y tiene muchos detalles; cada detalle pertenece a una variante. Esas relaciones se definen en `Producto`, `ProductoVariante`, `Pedido`, `PedidoDetalle` y `Cliente`.

7. El catálogo: consulta y transformación

`ProductoController::index()` atiende `GET /api/productos`. La consulta carga las variantes con el producto:

```
$query = Producto::query()->with('variantes');

if ($request->filled('categoria')) {
    $query->whereRaw(
        'LOWER(categoria) = ?',
        [mb_strtolower($request->input('categoria'))]
    );
}

$productos = $query->get();
```

`with('variantes')` hace carga anticipada para evitar pedir las variantes una por una. El `?` es un parámetro enlazado: el valor de categoría se envía por separado y no se concatena como SQL. `filled()` significa que el parámetro fue enviado y no está vacío.

Después, el controlador transforma cada producto a la estructura que espera el frontend. Las variantes incluyen talle, color, stock e imagen. También crea una lista resumida de colores y asigna colores e imágenes compatibles con el catálogo. `imageUrl()` mantiene URLs externas completas y convierte rutas locales en URLs servibles por Laravel (`ProductoController`).

Esta transformación explica por qué la respuesta JSON puede incluir campos que no son columnas, como `colors`, `es_nuevo` u `oferta`: son parte del contrato que se presenta al frontend, no necesariamente datos almacenados en una tabla.

Si falla la consulta, el controlador registra el detalle en el log del servidor y devuelve un mensaje genérico. El mensaje público evita exponer detalles internos de la excepción.

8. Ejemplo guiado: registrar un pedido

El pedido es el flujo principal que conecta validación, catálogo, CAPTCHA, transacciones y relaciones.

Paso 1: validar forma y tipos

`PedidoController::store()` empieza validando la estructura:

```
$validated = $request->validate([
    'cliente.nombre' => ['required', 'string', 'max:255'],
    'entrega' => ['required', 'in:retiro,domicilio'],
    'items' => ['required', 'array', 'min:1', 'max:40'],
    'items.*.id_variante' => [
        'required', 'integer', 'distinct',
        'exists:producto_variantes,id_variante',
    ],
    'items.*.cantidad' => ['required', 'integer', 'min:1', 'max:2147483647'],
    'captcha_challenge_id' => ['required', 'uuid'],
]);
```

`items.*` aplica una regla a cada elemento de la lista. `distinct` impide repetir el mismo identificador de variante en dos líneas; `exists` comprueba que ese identificador esté en la tabla. Para la entrega a domicilio, la dirección es obligatoria mediante `required_if`.

La validación de estructura no sustituye la comprobación de negocio: el controlador aún debe comprobar el stock vigente y calcular el total.

Paso 2: comprobar honeypot y CAPTCHA

Un campo llamado `website` funciona como honeypot: debería permanecer vacío. Luego `Cache::pull()` recupera y elimina la respuesta asociada al desafío. Eso hace que un CAPTCHA sea de un solo uso, incluso si se intenta repetir. El cálculo y su vencimiento están en `CaptchaController`.

El checkout del frontend envía el identificador y la respuesta directamente a `POST /api/pedidos`, que los verifica de nuevo dentro del flujo del pedido. `POST /api/captcha/verify` también está disponible para comprobar un desafío por separado, pero no es el paso que registra el pedido.

Paso 3: abrir una transacción y consultar variantes

Una transacción hace que varias escrituras se confirmen como una sola unidad: si ocurre una excepción antes de terminar, Laravel revierte los cambios de base de datos de esa transacción.

```
$order = DB::transaction(function () use ($validated): Pedido {
    $variantIds = collect($validated['items'])
        ->pluck('id_variante')
        ->sort()
        ->values();

    $variants = ProductoVariante::query()
        ->with('producto')
        ->whereIn('id_variante', $variantIds)
        ->orderBy('id_variante')
        ->lockForUpdate()
        ->get()
        ->keyBy('id_variante');

    // Verificar stock y calcular el precio del lado del servidor.
});
```

La lista ordenada ayuda a bloquear las variantes en un orden consistente. `lockForUpdate()` solicita bloquear las filas seleccionadas durante la transacción en los motores de base de datos que admiten ese bloqueo; esto reduce conflictos cuando varias operaciones intentan usar el mismo stock a la vez. El motor y su configuración deben ser adecuados para el uso concurrente real.

Paso 4: calcular precio y verificar existencias

Para cada artículo, el backend obtiene el precio del producto asociado con la variante. No confía en un total ni un precio enviados por el navegador:

```
foreach ($validated['items'] as $item) {
    $variant = $variants->get($item['id_variante']);

    if (! $variant || $variant->stock < $item['cantidad']) {
        throw ValidationException::withMessages([
            'items' => 'No hay suficiente stock para esta variante.',
        ]);
    }

    $unitPriceInCents = (int) round((float) $variant->producto->precio * 100);
    $totalInCents += $unitPriceInCents * $item['cantidad'];
}
```

Trabajar con centésimos enteros evita acumular errores binarios de coma flotante al sumar importes decimales. Antes de continuar se limita el total máximo permitido.

Paso 5: guardar cliente, pedido y detalles

Después de validar todos los artículos, la transacción crea:

1. Un cliente con nombre, teléfono y dirección o texto de retiro.
2. Un pedido con estado inicial `Pendiente` y total calculado.
3. Un detalle por variante, con cantidad y precio unitario vigente en ese momento.

El precio unitario del detalle es una instantánea histórica: si luego cambia el precio del catálogo, el pedido conserva el valor con el que se registró. Finalmente, se cargan las relaciones necesarias y se devuelve el pedido con estado HTTP `201 Created` (`PedidoController::store`).

Importante: registrar no equivale a reservar stock

Al crear el pedido, el stock se comprueba pero no se descuenta. El pedido queda pendiente y el inventario se reduce cuando la administración lo confirma. Por eso puede haber dos pedidos pendientes que, juntos, soliciten más unidades que las disponibles; al confirmar, la transacción vuelve a comprobar el stock y solo puede aceptar una cantidad que aún exista.

9. Estados del pedido y cambios de inventario

Los cambios permitidos siguen reglas específicas:

Estado actual	Nuevo estado	Efecto sobre stock
`Pendiente`	`Confirmado`	Verifica todas las variantes y descuenta las cantidades.
`Pendiente`	`Cancelado`	No cambia el stock, porque aún no se había descontado.
`Confirmado`	`Entregado`	No cambia el stock; el pedido ya lo descontó al confirmarse.
`Confirmado`	`Cancelado`	Devuelve las cantidades descontadas.
Cualquier otro cambio	—	Se rechaza como transición no permitida.

La confirmación bloquea el pedido y las variantes, vuelve a comprobar todas las cantidades y solo después descuenta. Si el mismo estado se solicita otra vez, retorna el pedido sin descontar nuevamente. La cancelación de un pedido confirmado también comprueba que devolver las unidades no supere el máximo aceptado por la columna de stock. La lógica está en `PedidoController::updateStatus`.

`orderData()` compone una respuesta estable para el frontend: reúne datos del pedido, cliente y detalles, incluyendo nombre de producto, talle y color. Las consultas usan `with(...)` para traer esas relaciones antes de transformarlas.

10. CAPTCHA y protección básica contra abuso

`CaptchaController::challenge()` genera dos números aleatorios, crea un UUID y guarda la suma en la caché durante cinco minutos. El cliente recibe el identificador y la pregunta, no la respuesta:

```
$firstNumber = random_int(1, 9);
$secondNumber = random_int(1, 9);
$challengeId = (string) Str::uuid();

Cache::put(
    'checkout-captcha:'.$challengeId,
    $firstNumber + $secondNumber,
    now()->addMinutes(5),
);
```

Al verificar, el backend consume la respuesta con `Cache::pull`. Una respuesta incorrecta, ya usada o vencida se rechaza; el campo honeypot también se debe mantener vacío. Las rutas de desafío, verificación, pedido, registro e inicio de sesión tienen límites de frecuencia en `routes/api.php`.

Esto es una capa básica de mitigación, no una garantía contra automatización. En producción, la caché debe persistir durante esos cinco minutos y ser compartida entre instancias si hay más de un servidor atendiendo la API.

11. Autenticación de administradores

Registro inicial

El endpoint de registro exige nombre, correo, contraseña confirmada y una clave configurada en el servidor. El registro queda cerrado cuando ya existe un administrador. La contraseña se transforma con `Hash::make`; la clave se compara con `hash_equals`:

```
if (! hash_equals($registrationKey, $validated['registration_key'])) {
    return response()->json(['message' => 'La clave no es válida.'], 403);
}

$administrator = Administrador::create([
    'nombre' => $validated['nombre'],
    'email' => strtolower($validated['email']),
    'contrasena' => Hash::make($validated['password']),
]);
```

Nunca pongas la clave de registro en el frontend ni en un repositorio público. Configúrala solo en el entorno privado del servidor con un valor aleatorio y difícil de adivinar. El registro responde `503` si la clave no está configurada y `403` si el valor enviado no coincide. Si ya hay una cuenta, la ruta rechaza un nuevo registro.

Inicio de sesión y token

El login busca el correo y verifica la contraseña con `Hash::check`. Cuando coinciden, Sanctum genera un token personal:

```
$token = $administrator->createToken(
    'admin-panel',
    ['admin'],
    now()->addHours(12)
)->plainTextToken;
```

`plainTextToken` es el valor que se entrega una sola vez al frontend para usar como bearer token. La base guarda la versión protegida. El token tiene capacidad `admin` y expiración configurada al crearlo.

Las rutas privadas pasan por `auth:sanctum` y por el middleware `abilities`, registrado en `bootstrap/app.php`. `auth:sanctum` comprueba el token; `abilities:admin` exige la capacidad correcta. Tener un token sin esa capacidad no alcanza. Al cerrar sesión, el controlador elimina el token actual.

`Administrador` usa el trait `HasApiTokens` de Sanctum y oculta `contrasena` al serializar el modelo (`Administrador`). La sesión del frontend guarda el token en el navegador; el backend sigue validándolo en cada solicitud protegida.

Administrador inicial

`ProductoSeeder` agrega datos de ejemplo y crea un administrador inicial solo si están configurados los valores de bootstrap requeridos en el entorno. En producción, protege esas credenciales, no las registres en Git y no vuelvas a ejecutar seeders sin entender qué registros insertan.

12. Crear y editar productos

Validación antes de guardar

`AdminProductoController::store()` valida nombre, precio, categoría, archivo y variantes. El precio debe ser mayor que cero; el archivo debe ser una imagen JPG, PNG o WebP de hasta 5 MB; debe existir al menos una variante y cada stock debe ser un entero no negativo.

También se evita repetir la misma combinación de talle y color. Se recortan espacios, se pasa el texto a minúsculas y se compara la combinación resultante:

```
$combinations = array_map(
    fn (array $variant) => mb_strtolower(
        trim($variant['talle']).'|'.trim($variant['color'])
    ),
    $validated['variantes'],
);

if (count($combinations) !== count(array_unique($combinations))) {
    throw ValidationException::withMessages([
        'variantes' => 'No repitas la combinación de talle y color.',
    ]);
}
```

El separador `|` permite formar una clave simple con dos campos; normalizar antes de comparar hace que diferencias de mayúsculas o espacios no creen combinaciones duplicadas.

Guardar la imagen y la transacción

Si llega `imagen_archivo`, Laravel la almacena en el disco público, dentro de `productos/`. El registro del producto y sus variantes se guarda en una transacción:

```
$product = DB::transaction(function () use ($validated, $imagePath) {
    $product = Producto::create([
        'nombre' => $validated['nombre'],
        'precio' => $validated['precio'],
        'categoria' => $validated['categoria'],
        'imagen' => $imagePath,
    ]);

    $product->variantes()->createMany($validated['variantes']);

    return $product->load('variantes');
});
```

Si la operación de base de datos falla después de subir el archivo, el controlador borra ese archivo para no dejar una imagen huérfana. La base de datos y el sistema de archivos no comparten una transacción automática; este borrado compensatorio es la forma de limpiar el archivo cuando el guardado principal falla. Para servir `/storage/...`, se crea el enlace público:

```
php artisan storage:link
```

Editar sin destruir el historial

La edición usa una transacción y bloquea el producto y sus variantes antes de aplicar cambios. Cada `id_variante` recibido debe pertenecer a ese mismo producto. Las nuevas variantes no tienen identificador; las existentes sí.

Una variante asociada a un detalle de pedido no se puede borrar ni cambiar de talle o color. Así, los pedidos anteriores siguen apuntando a la opción que realmente se vendió. Si ya no se ofrece, se conserva con stock cero y se puede agregar una nueva variante. La prueba `test_product_update_preserves_variants_referenced_by_orders` comprueba esta protección.

Al reemplazar una imagen almacenada localmente, el código guarda la nueva, actualiza el producto y luego elimina el archivo anterior solo si reconoce que pertenece a la carpeta esperada. Si falla la transacción, elimina la imagen nueva y preserva la existente. Las URLs remotas no se borran del disco local. El flujo está en `AdminProductoController`.

13. Cambiar stock de una variante

La ruta `PATCH /api/admin/variantes/{variantId}/stock` acepta un entero entre cero y el máximo de la columna:

```
$validated = $request->validate([
    'stock' => ['required', 'integer', 'min:0', 'max:2147483647'],
]);

$variant = ProductoVariante::query()->findOrFail($variantId);
$variant->update(['stock' => $validated['stock']]);
```

Permitir cero significa “agotado”; no se elimina la variante. Esto es importante si aparece en pedidos anteriores. La operación requiere token de administrador por la agrupación de rutas, no solo por validación del controlador. La prueba de stock cero está en `AdminManagementTest`.

14. Migraciones y datos iniciales

Una migración es un cambio versionado del esquema. `up()` aplica el cambio y `down()` lo revierte cuando es posible:

```
Schema::create('productos', function (Blueprint $table) {
    $table->id('id_producto');
    $table->string('nombre');
    $table->decimal('precio', 10, 2);
    $table->string('categoria');
    $table->timestamps();
});
```

Las claves foráneas mantienen relaciones entre tablas. Por ejemplo, `producto_variantes.id_producto` apunta a productos y define qué ocurre al eliminar un producto. Revisa las migraciones antes de modificar tablas con datos reales; no edites una migración ya aplicada como sustituto de una nueva.

`php artisan migrate` ejecuta las migraciones pendientes. `php artisan db:seed` llama a `DatabaseSeeder`, que a su vez llama al seeder de productos. Ese seeder inserta datos de ejemplo; úsalo con una base de desarrollo limpia o revisa su contenido antes de repetirlo.

15. Configuración para desarrollo

En PowerShell, desde la carpeta del backend:

```
composer install
Copy-Item .env.example .env
php artisan key:generate
php artisan migrate
php artisan storage:link
php artisan serve
```

Antes de migrar, revisa el `.env` local:

- `APP_KEY` debe generarse para esa instalación.
- `DB_CONNECTION` y demás valores de base de datos deben corresponder al servidor local. El ejemplo del proyecto usa SQLite.
- `CORS_ALLOWED_ORIGINS` debe incluir el origen exacto del frontend de desarrollo, normalmente `http://localhost:5173` y/o `http://127.0.0.1:5173`.
- `ADMIN_REGISTRATION_KEY` debe tener un valor privado y aleatorio. No uses valores de ejemplo en producción.
- Para crear un administrador mediante el seeder se configuran sus valores de bootstrap solo en un entorno seguro.

Después de cambiar variables de entorno, Laravel puede mantener configuración en caché. En desarrollo, `php artisan config:clear` hace que vuelva a leer la configuración. No muestres ni compartas el contenido de `.env`.

Para inspeccionar rutas y ejecutar las pruebas:

```
php artisan route:list --path=api
php artisan test
```

`phpunit.xml` configura las pruebas para usar SQLite en memoria y una caché aislada. Por eso los tests no deberían modificar la base persistente de desarrollo.

16. CORS: por qué el navegador puede bloquear una solicitud

Un frontend en `localhost:5173` y un backend en `localhost:8000` tienen orígenes distintos porque cambia el puerto. CORS es una regla del navegador que pregunta al servidor si ese origen está autorizado.

`config/cors.php` permite métodos y encabezados, pero limita los orígenes a `CORS_ALLOWED_ORIGINS`. Si una llamada falla en el navegador, aunque el endpoint exista, revisa la consola y confirma que el origen esté escrito con el protocolo y puerto exactos. CORS no autentica al administrador ni reemplaza middleware de autorización.

17. Cómo se prueban los flujos

Las pruebas de características ejercitan rutas HTTP y verifican tanto respuestas como base de datos. `RefreshDatabase` prepara un esquema limpio para cada caso, y métodos como `postJson()` simulan peticiones JSON:

```
$response = $this->postJson('/api/pedidos', $payload);

$response->assertCreated()
    ->assertJsonPath('order.estado', 'Pendiente')
    ->assertJsonPath('order.total', '251.00');

$this->assertDatabaseCount('pedidos', 1);
```

La prueba también puede enviar un total falso para comprobar que el backend usa el precio del catálogo. No es suficiente verificar solo un `200`: se comprueban los valores persistidos y, en rutas protegidas, qué identidad o capacidad tiene el token.

Las pruebas de este proyecto cubren, entre otras cosas:

- Registro, inicio de sesión, permisos, límites y cierre del registro inicial.
- Creación y edición de productos, imágenes, variantes y stock agotado.
- Protección de variantes que forman parte del historial de pedidos.
- Precios calculados en el servidor y desafíos CAPTCHA de un solo uso.
- Confirmar, cancelar y entregar pedidos, y descontar o devolver stock una sola vez.
- Rechazo de rutas de prueba que no deberían estar disponibles.

Consulta `OrderManagementTest`, `AdminManagementTest`, `CaptchaTest` y `SecurityHardeningTest`.

18. Errores comunes al desarrollar

- **El frontend no llega a la API:** confirma que `php artisan serve` esté activo, que la URL base incluya `/api` y que CORS permita el origen.
- **Error `401`:** falta un token válido o expiró.
- **Error `403`:** la operación no tiene la capacidad requerida, el registro está cerrado o la clave administrativa no coincide.

- **Error `422`:** revisa los errores de validación que devuelve Laravel; puede ser un campo inválido, stock insuficiente o una transición no permitida.
- **Error `429`:** se superó el límite temporal de solicitudes.
- **No se ve una imagen cargada:** revisa que exista el enlace creado por `storage:link` y que la ruta pública corresponda al disco configurado.
- **Un pedido no descuenta stock inmediatamente:** es intencional; el pedido se registra pendiente y la confirmación administrativa descuenta las existencias.
- **Una variante antigua no se puede borrar:** si tiene historial de pedido, se conserva para que los detalles históricos sigan teniendo significado.

19. Orden recomendado para estudiar el código

1. Lee `routes/api.php` para aprender qué operaciones hay.
2. Sigue `GET /api/productos` hasta `ProductoController`.
3. Lee `Producto` y `ProductoVariante` para entender las relaciones.
4. Sigue `POST /api/pedidos` en `PedidoController`, relacionándolo con `Pedido`, `PedidoDetalle` y `Cliente`.
5. Dibuja las transiciones de pedido y marca en qué paso cambia el stock.
6. Lee `CaptchaController` y luego `AdminAuthController`.
7. Termina con `AdminProductoController` y sus pruebas.

Para saltar a una línea en VS Code, abre un archivo y pulsa **Ctrl+G**. Para entender una función, intenta responder tres preguntas: qué recibe, qué comprueba y qué devuelve o modifica.

20. Glosario

Término	Significado
Endpoint	Combinación de método HTTP y ruta que realiza una operación.
Middleware	Capa que procesa una solicitud antes de llegar al controlador.
Controlador	Clase que coordina validación, reglas y respuesta HTTP.
Modelo	Clase Eloquent que representa una tabla o una relación.
Migración	Cambio versionado de la estructura de la base de datos.
Seeder	Código que inserta datos iniciales o de ejemplo.
Transacción	Grupo de operaciones de base de datos que se confirman o revierten juntas.
Clave foránea	Columna que conecta un registro con otro y protege esa relación.
Token bearer	Credencial que el cliente envía en el encabezado Authorization.
Capacidad / ability	Permiso asociado a un token de Sanctum.
Caché	Almacenamiento temporal; aquí conserva la respuesta del CAPTCHA.
Eager loading	Cargar relaciones junto con el modelo para evitar consultas repetidas.
Honeypot	Campo trampa que un formulario humano deja vacío y un bot puede completar.

Término	Significado
CORS	Política del navegador para llamadas entre orígenes distintos.

21. Inventario de archivos: cobertura de esta guía

Este inventario sirve para comprobar qué partes del backend se explican. Se enumeran los archivos propios que definen el comportamiento, la configuración, las pruebas o el proceso de desarrollo. Las rutas relativas abren los archivos originales; los archivos binarios y las salidas locales se excluyen al final.

Rutas, controladores, modelos y arranque

Archivo	Responsabilidad
`routes/api.php`	Contrato HTTP: catálogo, CAPTCHA, pedidos, registro e inicio de sesión; agrupa rutas privadas con `auth:sanctum` y la capacidad `admin`, y limita solicitudes sensibles.
`routes/web.php`	Rutas web tradicionales; expone la página inicial de Laravel separada de la API.
`routes/console.php`	Punto para definir comandos de consola programados o personalizados.
`app/Http/Controllers/Controller.php`	Clase base vacía que permite compartir comportamiento entre controladores.
`app/Http/Controllers/ProductoController.php`	Consulta el catálogo y transforma producto/variantes a la respuesta pública que consume Vue, incluyendo URLs de imágenes.
`app/Http/Controllers/Api/PedidoController.php`	Valida y registra pedidos, recalcula precios, consume CAPTCHA, lista pedidos de administración y aplica transiciones de estado con efectos de stock.
`app/Http/Controllers/Api/CaptchaController.php`	Genera desafíos temporales, los consume una sola vez y rechaza respuestas incorrectas o un honeypot completado.
`app/Http/Controllers/Api/AdminAuthController.php`	Cierra el registro tras crear el primer administrador; valida credenciales, emite token Sanctum de duración limitada y revoca el token actual al salir.
`app/Http/Controllers/Api/AdminProductoController.php`	Actualiza stock; crea/edita producto, variantes e imagen. Protege identificadores de variante y el historial de pedidos durante la edición.
`app/Models/Producto.php`	Define tabla, clave, campos asignables y relación uno-a-muchos con variantes.
`app/Models/ProductoVariante.php`	Representa talle/color/stock e imagen de variante; pertenece a un producto y se enlaza con detalles de pedido.
`app/Models/Cliente.php`	Representa la información de contacto y entrega; tiene pedidos.
`app/Models/Pedido.php`	Representa estado, entrega, fecha, total y notas; pertenece a cliente y contiene detalles.
`app/Models/PedidoDetalle.php`	Guarda variante, cantidad y precio unitario histórico de una línea de pedido.
`app/Models/Administrador.php`	Modelo autenticable de la tabla singular `administrador`; protege la contraseña y crea tokens Sanctum.
`app/Models/User.php`	Modelo estándar del esqueleto Laravel para la tabla `users`; no representa el acceso administrativo propio de esta tienda.

Archivo	Responsabilidad
`app/Providers/AppServiceProvider.php`	Proveedor de servicios de aplicación; actualmente no registra personalizaciones.
`bootstrap/app.php`	Ensambla rutas y middleware; configura cómo serializar errores de API como JSON.
`bootstrap/providers.php`	Lista los proveedores de servicios cargados por la aplicación.
`public/index.php`	Entrada HTTP: carga el arranque de Laravel y entrega la petición al kernel de la aplicación.
`public/.htaccess`	Reglas de Apache para dirigir solicitudes al punto de entrada público.
`public/robots.txt`	Indicaciones para rastreadores web; no implementa reglas de autorización.

Esquema, datos de ejemplo y pruebas

Archivo(s)	Qué explican
`database/migrations/0001_01_01_000000_create_users_table.php`, `0001_01_01_000001_create_cache_table.php` y `0001_01_01_000002_create_jobs_table.php`	Tablas de infraestructura del esqueleto Laravel: usuarios/sesiones y restablecimiento de contraseña, caché/bloqueos, colas, lotes y trabajos fallidos.
`2026_09_25_032643_create_productos_table.php` y `2026_09_25_032644_create_producto_variantes_table.php`	Catálogo y opciones por talle/color; las variantes son las unidades a las que se asigna stock.
`2026_09_25_032645_create_clientes_table.php`, `2026_09_25_032646_create_pedidos_table.php` y `2026_09_25_032647_create_pedido_detalles_table.php`	Cliente, encabezado del pedido y líneas de artículos con cantidades y precios históricos.
`2026_09_25_033012_create_personal_access_tokens_table.php` y `2026_09_25_073424_create_administrador_table.php`	Tokenización de Sanctum y tabla propia de administradores.
`2026_09_29_180000_fix_seeded_product_images.php` y `2026_09_29_183000_use_classic_bra_asset.php`	Ajustes de rutas de imágenes en registros ya sembrados; no crean tablas.
`2026_10_01_031448_add_imagen_to_producto_variantes_table.php` y `2026_11_02_122913_add_notas_to_pedidos_table.php`	Cambios incrementales para imagen propia de variante y notas opcionales del pedido.
`database/.gitignore`	Conserva la carpeta de base de datos en Git sin incluir los archivos SQLite generados localmente.
`database/seeders/DatabaseSeeder.php`	Punto de entrada de `db:seed`; llama a los seeders de productos y puede preparar administrador solo con configuración de bootstrap disponible.
`database/seeders/ProductoSeeder.php`	Carga productos y variantes de demostración, con imágenes servidas desde el backend. Revisarlo antes de ejecutarlo en una base con datos reales.
`database/factories/UserFactory.php`	Genera usuarios ficticios del modelo Laravel para pruebas y datos de desarrollo.
`tests/TestCase.php`	Clase base de pruebas; hereda el entorno de pruebas de Laravel.
`tests/Feature/AdminManagementTest.php`	Registro/login, permisos, creación/edición, identificadores, variantes, imágenes y stock, incluidos casos de error e historial.
`tests/Feature/OrderManagementTest.php`	Precio fijado por servidor, pedido pendiente, CAPTCHA reutilizado, confirmación/cancelación y escenarios de falta de stock/autorización.
`tests/Feature/CaptchaTest.php`	Aceptación de respuesta una sola vez, rechazo de error/honeypot y límite de solicitudes.

Archivo(s)	Qué explican
`tests/Feature/SecurityHardeningTest.php`	Ausencia de rutas de prueba antiguas, catálogo con variantes, cierre del registro y capacidad administrativa.
`tests/Feature/ExampleTest.php` y `tests/Unit/ExampleTest.php`	Pruebas de ejemplo del esqueleto: una respuesta HTTP y una aserción unitaria básica; no prueban reglas del negocio.
`phpunit.xml`	Descubre suites Unit/Feature y configura SQLite en memoria, caché aislada y otros valores de entorno de prueba.

Qué escenarios específicos están cubiertos

Los siguientes casos identifican qué leer cuando se estudia el comportamiento; los nombres son los métodos reales de las pruebas:

Archivo y método(s)	Afirmación que verifica
`OrderManagementTest`: `test_customer_can_register_a_pending_order_using_server_prices`	Pedido público con precio del catálogo, estado pendiente y persistencia de relaciones/detalles.
`OrderManagementTest`: `test_order_creation_rejects_insufficient_stock_and_reused_captcha`, `test_admin_confirmation_deducts_stock_and_cancellation_restores_it_once`, `test_admin_cannot_confirm_a_pending_order_after_stock_runs_out`, `test_order_management_requires_an_administrator_token`	CAPTCHA de un solo uso, falta de stock, descuento al confirmar, devolución única al cancelar, nueva comprobación de existencias y permiso de administración.
`AdminManagementTest`: `test_registration_requires_the_configured_admin_key`, `test_admin_can_register_login_and_create_a_product_with_variants`, `test_product_creation_requires_an_authenticated_admin`, `test_product_creation_rejects_missing_database_fields`, `test_product_update_requires_an_authenticated_admin`	Clave de registro configurada, hash/token de sesión, validación de campos obligatorios y autenticación requerida para crear o editar.
`AdminManagementTest`: `test_admin_can_update_variant_stock_to_zero`, `test_stock_update_requires_an_admin_and_rejects_negative_values`	Cero permitido como agotado; valores negativos y usuarios no autorizados rechazados.
`AdminManagementTest`: `test_admin_can_upload_a_product_image`, `test_admin_can_replace_and_remove_a_product_image`	Subida, reemplazo y limpieza de imágenes mediante el disco público falso de pruebas.
`AdminManagementTest`: `test_admin_can_update_a_product_and_its_variants`, `test_product_update_rejects_variant_ids_from_another_product`, `test_product_update_rejects_duplicate_variant_ids`, `test_product_update_preserves_variants_referenced_by_orders`	Edición de variantes existentes/nuevas, propiedad del identificador, duplicados y protección de referencias históricas.
`CaptchaTest`: `test_correct_answer_is_accepted_only_once`, `test_wrong_answer_and_honeypot_are_rejected`, `test_challenge_endpoint_limits_requests_per_minute`	Consumo de desafío, respuesta inválida, honeypot y rate limit.
`SecurityHardeningTest`: `test_the_public_test_order_route_no_longer_exists`, `test_the_image_generation_routes_no_longer_exist`, `test_product_listing_works_with_variants`, `test_admin_registration_is_closed_once_an_administrator_exists`, `test_admin_routes_require_the_admin_ability`	Endpoints de prueba eliminados, catálogo con variantes, registro cerrado tras el primer administrador y autorización con capacidad `admin`.

La cobertura de una prueba indica qué comportamiento se ejercitó, no que todos los posibles estados de producción estén probados. Ejecuta `php artisan test` para confirmar el resultado con el entorno local actual.

Configuración, recursos y herramientas del repositorio

Archivo(s)	Función
`config/admin.php`	Lee la clave de registro administrativo desde la configuración del servidor. No copies el valor de desarrollo del archivo de ejemplo a esta guía o al cliente web.
`config/app.php`, `config/auth.php`, `config/cache.php`, `config/cors.php`	Nombre/entorno, guardas de autenticación, almacenamiento temporal y orígenes/métodos permitidos por CORS.
`config/database.php`, `config/filesystems.php`, `config/logging.php`	Conexiones de base de datos, discos de imágenes y canales de registro.
`config/mail.php`, `config/queue.php`, `config/sanctum.php`	Correo, trabajos en cola y configuración de tokens/autenticación de Sanctum.
`config/services.php`, `config/session.php`	Configuración para integraciones externas y sesiones web.
`resources/views/welcome.blade.php`	Vista de bienvenida que sirve la ruta web inicial; no es la interfaz Vue ni la respuesta del catálogo API.
`resources/css/app.css`, `resources/js/app.js`	Puntos de entrada Vite del esqueleto Laravel. No contienen la tienda Vue, que vive en el repositorio frontend.
`public/.htaccess`, `public/index.php`, `public/robots.txt`	Reglas del servidor web, entrada HTTP de Laravel y directiva para rastreadores, respectivamente.
`composer.json`	Requisitos PHP, dependencias, carga PSR-4 y tareas de Composer para preparación, desarrollo y pruebas.
`package.json`, `vite.config.js`	Paquetes y compilación de los recursos Vite del esqueleto Laravel; no son el `package.json` del frontend independiente.
`artisan`	Lanzador de comandos de Laravel, por ejemplo `route:list`, `migrate` y `test`.
`.env.example`	Plantilla de configuración para una instalación local. Contiene valores de ejemplo; este documento deliberadamente no reproduce sus valores ni ningún contenido del `.env` real.
`README.md`	Descripción de API de administración, condiciones para habilitarla, imágenes y enlaces de estudio.
`AGENTS.md`, `CLAUDE.md`	Instrucciones de trabajo para asistentes de código; no se ejecutan como parte de una solicitud HTTP.
`.editorconfig`, `.gitattributes`, `.gitignore`, `.npmrc`	Convenciones de editor, tratamiento de archivos en Git, exclusiones locales y opciones del cliente npm.
`.mcp.json`, `boost.json`	Configuración para herramientas de desarrollo/agentes; no contiene endpoints públicos de la API de la tienda.
`routes/api.php`	Es el contrato real de las rutas API. Para ver su registro efectivo, compáralo con `php artisan route:list --path=api`.

Instrucciones de desarrollo agrupadas

Los siguientes archivos describen procedimientos para asistentes o colaboradores, no componentes que Laravel cargue al atender pedidos:

- `.claude/skills/deploying-to-cloud/SKILL.md` y `.claude/skills/deploying-to-cloud/reference/checklists.md`: pasos de despliegue y su lista de control.
- `.claude/skills/infer-conventions/SKILL.md` y `.claude/skills/infer-conventions/references/checklist.md`: guía para inferir convenciones existentes antes de tocar el código.

- `.claude/skills/laravel-best-practices/SKILL.md` y sus reglas `advanced-queries.md`, `architecture.md`, `blade-views.md`, `caching.md`, `collections.md`, `config.md`, `db-performance.md`, `eloquent.md`, `error-handling.md`, `events-notifications.md`, `http-client.md`, `mail.md`, `migrations.md`, `queue-jobs.md`, `routing.md`, `scheduling.md`, `security.md`, `style.md` y `validation.md`: referencias de prácticas Laravel por tema.
- `.claude/skills/tailwindcss-development/SKILL.md`: instrucciones para desarrollo con Tailwind, una herramienta que no construye la UI Vue de la tienda.
- `.claude/skills/testing-best-practices/SKILL.md` y sus reglas `assertions.md`, `endpoint-tests.md`, `finding-features.md`, `isolation.md`, `naming.md`, `performance.md`, `review.md`, `security.md` y `test-data.md`: pautas para diseñar y revisar pruebas.

No forman parte de esta lectura los archivos de `.env`, la base SQLite local, `vendor/`, `node_modules/`, `bootstrap/cache/`, `storage/` generado, la caché de resultados PHPUnit, los lockfiles ni los binarios de `public/images/` y `public/favicon.ico`. Las imágenes son activos que puede servir la API, pero no código fuente legible. Se excluyen también esta guía y el PDF para evitar contarlos como parte del código documentado.

Recorrido anotado: dónde se decide un cambio de stock

```
if ($order->estado === 'Pendiente' && $nextStatus === 'Confirmado') { // (1) solo esta transición reserva existencias
    $variants = $this->lockOrderVariants($order); // (2) bloquea las variantes del pedido en orden estable
    foreach ($order->detalles as $detail) { // (3) valida todas antes de modificar alguna
        $variant = $variants->get($detail->id_variante); // (4) busca la variante ya bloqueada
        if (! $variant || $variant->stock < $detail->cantidad) { // (5) si una no alcanza, la transacción falla
            throw ValidationException::withMessages(['estado' => 'Stock insuficiente.']);
        }
    }
    foreach ($order->detalles as $detail) { // (6) recién después descuenta cada cantidad
        ProductoVariante::query()
            ->whereKey($detail->id_variante)
            ->decrement('stock', $detail->cantidad);
    }
    $order->update(['estado' => $nextStatus]); // (7) confirma estado y stock en la misma transacción
}
```

El código real también rechaza una variante inexistente y verifica el resultado de cada descuento; aquí se conserva la secuencia para que se vea el punto de decisión. El pedido recién creado queda `Pendiente`; cambiarlo a `Confirmado` es el paso que descuenta stock. Cancelar un pendiente no lo modifica. Cancelar un confirmado lo devuelve una sola vez, mientras pasar de confirmado a entregado no vuelve a tocarlo.

22. Lectura comentada, archivo por archivo

El inventario del capítulo anterior permite localizar todo el alcance; este capítulo explica qué hace cada grupo de archivos en el recorrido real del programa. Léelo con el IDE abierto: las referencias apuntan a responsabilidades y símbolos reales, mientras que los ejemplos muestran el patrón de control sin copiar archivos enteros.

Punto de entrada, arranque y rutas

`public/index.php`: entrada de cada petición web

El servidor web dirige aquí las solicitudes. El archivo define el modo de arranque, carga el autoloader de Composer, incluye `bootstrap/app.php`, crea una petición HTTP desde los datos recibidos y la entrega al manejador de Laravel. El manejador produce una respuesta y la envía; finalmente se ejecutan las tareas de cierre. No valida productos ni consulta tablas directamente: su función es conectar el servidor con el framework.

`bootstrap/app.php`: ensamblar rutas, middleware y excepciones

`Application::configure(basePath: ...)` crea la aplicación con la raíz del repositorio. `withRouting()` conecta por separado `routes/web.php`, `routes/api.php` y `routes/console.php`, y declara `/up` como comprobación de salud. `withMiddleware()` registra el alias `abilities` para el middleware de Sanctum `CheckAbilities`; ese alias permite expresar `abilities:admin` en las rutas privadas. `withExceptions()` hace que Laravel renderice como JSON los errores bajo `api/*` (o los que pidan JSON), comportamiento importante para que Vue pueda interpretar errores `401`, `403` y `422`.

El alias solo da un nombre corto a la clase; no concede permiso por sí solo. La ruta también necesita autenticación y el token debe tener la capacidad solicitada.

`bootstrap/providers.php` y `app/Providers/AppServiceProvider.php`

`providers.php` devuelve la lista de proveedores adicionales que Laravel registra al arrancar. La lista actual incorpora `AppServiceProvider`. `register()` y `boot()` son los momentos estándar en los que un proveedor puede registrar servicios o personalizaciones; ambos están vacíos en `AppServiceProvider.php`. El archivo existe por convención, pero actualmente no contiene lógica de la tienda.

`routes/api.php`: declarar el contrato de la API

Laravel agrega el prefijo `/api` a estas rutas. Cada `Route::get/post/patch` asocia un método HTTP, una URI relativa y una acción `Controlador::método`. Los endpoints públicos de CAPTCHA, catálogo y pedido tienen límites de solicitudes según su middleware `throttle`. Dentro del prefijo `admin`, login y registro son públicos pero limitados; el grupo interior exige `auth:sanctum` y `abilities:admin` antes de llamar a cualquier acción protegida.

```
Route::post('/pedidos', [PedidoController::class, 'store'])
    ->middleware('throttle:5,1'); // (1) crea pedidos con un límite temporal

Route::middleware(['auth:sanctum', 'abilities:admin'])->group(function () {
    Route::patch('/variantes/{variantId}/stock', [AdminProductoController::class, 'updateStock']);
}); // (2) solo llega a updateStock con token y capacidad válidos
```

`{variantId}` es un parámetro de la URL que Laravel convierte en argumento para el método. La ruta de edición de producto restringe `{producto}` a números mediante `whereNumber`; las otras operaciones protegidas incluyen salir, crear producto, leer pedidos y cambiar el estado del pedido. Si una ruta no aparece aquí, no forma parte del contrato efectivo; `php artisan route:list --path=api` permite contrastar los registros en ejecución.

`routes/web.php` y `routes/console.php`

`web.php` define solo `/`, que renderiza la vista `welcome`. No sirve el catálogo: ese dato se obtiene de `GET /api/productos`. `console.php` está preparado para declarar comandos Artisan propios; actualmente no agrega comandos de negocio. Ambos archivos son entradas de routing distintas de la API.

Controladores: recibir, validar, coordinar, responder

`app/Http/Controllers/Controller.php`

La clase base extiende el controlador de Laravel y no añade métodos. Los controladores de la tienda heredan de ella para seguir la estructura habitual del framework. Al no contener reglas propias, modificarla no es una forma de cambiar cómo se calculan precios o permisos.

`app/Http/Controllers/ProductoController.php`

`index(Request $request)` es el endpoint público de lectura:

1. Crea una consulta Eloquent con `Producto::query()->with('variantes')`; cargar la relación de antemano evita una consulta por producto.
2. Si el filtro `categoria` viene completo, lo compara en minúsculas mediante SQL parametrizado (`?`). No concatena la entrada del usuario en el SQL.
3. Convierte cada modelo a una forma JSON destinada al frontend. El array de `variantes` conserva `id_variante`, talle, color, stock e imagen resuelta. `colors` quita colores repetidos, asigna un valor hexadecimal conocido y, para algunos productos antiguos, elige una imagen compatible por color.
4. Añade campos calculados `es_nuevo` y `oferta`, aunque no sean columnas almacenadas. Así se construye un contrato de presentación independiente del esquema SQL.
5. Si una excepción impide consultar el catálogo, escribe el detalle en el log del servidor y devuelve un mensaje genérico con HTTP 500.

`imageUrl(?string $image)` devuelve `null` si no hay imagen, deja intactas URLs HTTP(S) y convierte una ruta local, como `/images/...`, en una URL absoluta del host Laravel. Se utiliza tanto para la imagen del producto como para la variante, por eso el frontend puede tratar ambos tipos con el mismo campo `imagen_url`.

`app/Http/Controllers/Api/CaptchaController.php`

- `challenge()` obtiene dos números con `random_int`, crea un UUID y guarda la suma en Cache por cinco minutos. Responde únicamente con el ID y la pregunta, nunca con la suma esperada.
- `verify(Request $request)` valida UUID, respuesta entera y campo honeypot. Si el honeypot contiene texto, rechaza; luego usa `Cache::pull()`, que lee y elimina la solución en una operación. La respuesta correcta solo vale una vez y solo mientras no haya caducado.
- `rejectedResponse()` centraliza la respuesta JSON `422` para desafío incorrecto, inexistente, vencido o repetido.

La creación de pedidos no depende de que el navegador haya llamado primero a `verify`: `PedidoController::store()` también consume y comprueba el CAPTCHA en la misma solicitud que registra el pedido. De este modo, saltarse la interfaz no evita la validación del servidor.

`app/Http/Controllers/Api/AdminAuthController.php`

- `register()` primero consulta si ya existe un administrador. Si sí, cierra el registro antes de validar o crear otra cuenta. Luego valida correo único, contraseña confirmada y longitud, y exige una clave de registro presente en configuración. `hash_equals()` compara la clave evitando comparaciones de cadenas de tiempo variable. `Hash::make()` almacena una derivación de la contraseña, nunca el texto recibido.
- `login()` normaliza el correo a minúsculas, obtiene el administrador y utiliza `Hash::check()` para comparar contraseña contra hash. Una combinación incorrecta recibe el mismo mensaje genérico; no revela si el correo existe.
- `authenticatedResponse()` crea un token Sanctum con la capacidad `admin`, devuelve su texto al navegador una sola vez y acompaña una representación pública acotada del usuario. La duración configurada del token es de 12 horas; la base de datos guarda su hash, no el token original.
- `logout()` borra el token que acompaña la solicitud. Responde `204`, sin cuerpo JSON.

La clave de registro y los valores bootstrap se leen desde `config/admin.php`. La guía explica los nombres de configuración y el flujo, no reproduce valores del archivo de entorno ni de `.env.example`.

`app/Http/Controllers/Api/AdminProductoController.php`

Esta clase concentra tres operaciones y el cuidado del archivo asociado:

- `updateStock(Request $request, int $variantId)` valida que `stock` sea un entero no negativo dentro del límite de la columna, busca por clave primaria (`findOrFail`) y actualiza la variante. Devuelve el identificador y número confirmado por Eloquent. Un stock de cero representa agotado y no elimina la identidad histórica de la variante.
- `store()` valida nombre, descripción, precio decimal positivo, categoría, referencia de imagen, archivo (JPG/PNG/WebP hasta 5 MB) y lista de 1–40 variantes. Después normaliza la combinación talle/color para rechazar duplicados. Si recibe archivo, lo guarda en disco `public`; inserta producto y variantes dentro de `DB::transaction()`. Si falla la escritura, borra el archivo recién almacenado antes de propagar el error. La respuesta `201` devuelve el producto y variantes, con una URL de imagen lista para servir.
- `update(Request $request, Producto $producto)` recibe un modelo resuelto por route-model binding, y valida también IDs opcionales de variante: cada ID debe pertenecer a ese mismo producto y no repetirse. Dentro de transacción bloquea producto/variantes, calcula cuáles se quitaron y comprueba si alguna tiene referencias en `pedido_detalles`. Asimismo impide cambiar talle/color de una variante vendida. Conserva IDs que ya existen, crea las variantes sin ID y borra solo las retiradas que no tienen historial. El servidor no confía en el ID que la interfaz dibujó.
- La imagen nueva se guarda antes de modificar SQL. Si la transacción falla, se limpia el archivo nuevo; después de guardar con éxito se elimina la imagen local anterior únicamente si `storedProductImagePath()` confirma que la ruta corresponde a `storage/productos/<archivo>`. URLs remotas y rutas ajenas no se borran accidentalmente.
- `storedProductImagePath()` analiza la URL, rechaza barras invertidas, subrutas inesperadas y nombres `.` o `..`; solo devuelve un path relativo de la carpeta de imágenes administradas.

La base de datos y el disco no participan en una misma transacción atómica. El código compensa errores borrando el archivo temporal, y conserva la imagen anterior hasta que la nueva escritura SQL termina.

`app/Http/Controllers/Api/PedidoController.php`

El controlador combina el endpoint de compra y el panel administrativo:

- `store(Request $request)` valida cliente, entrega, dirección condicional, notas, lista de ítems, IDs distintos/existentes, cantidades y campos CAPTCHA. Rechaza el honeypot y consume el desafío con `Cache::pull()`.
- La transacción ordena los IDs de variante, carga productos y bloquea filas. Para cada línea busca la variante vigente, verifica stock y calcula el total a partir del precio real del producto. Convierte precios a centésimos enteros para limitar errores de coma flotante y prevenir desbordes.
- Solo cuando todas las líneas son válidas crea cliente, pedido `Pendiente` y detalles con cantidad y precio unitario actual. El stock no se descuenta en este paso. Devuelve HTTP `201` con `orderData()`.
- `adminIndex()` trae relaciones cliente/detalle/variante/producto y ordena por fecha descendente. `with(...)` reduce consultas repetidas.
- `updateStatus()` valida estados destino y bloquea el pedido en transacción. Repetir el estado actual es idempotente. Pendiente→Confirmado comprueba todas las existencias antes de descontar; Pendiente→Cancelado no afecta stock; Confirmado→Entregado no lo vuelve a modificar; Confirmado→Cancelado devuelve existencias una vez. Cualquier transición restante produce error de validación y revierte cambios.
- `lockOrderVariants()` bloquea en orden estable las variantes implicadas, para que transacciones concurrentes compitan sobre las mismas filas de forma más predecible.

- `orderData()` normaliza la salida pública: id, fecha, estado, total, entrega, notas, contacto y cada detalle con producto/talle/color. Así el cliente no depende de serializar arbitrariamente todos los atributos internos Eloquent.

Una transacción mantiene juntas las escrituras SQL: si se rechaza una línea, no queda un cliente o pedido parcial. Los bloqueos reducen carreras, pero la garantía final depende del motor y del aislamiento de base de datos utilizado en producción.

Modelos: traducir tablas a relaciones y campos seguros

Los modelos viven en `app/Models`. Como las tablas y claves del dominio usan nombres españoles no convencionales, declaran `$table` y `$primaryKey` en vez de depender de los nombres plurales ingleses predeterminados.

Archivo	Campos y conexiones importantes
`Producto.php`	Representa `productos`; `$fillable` limita asignación masiva a nombre, descripción, precio, categoría e imagen. `variantes()` es `hasMany(ProductoVariante)` con `id_producto` como clave local y externa.
`ProductoVariante.php`	Representa la unidad vendible en `producto_variantes`, con producto, talle, color, stock e imagen. `producto()` es `belongsTo`; pedidos apuntan a la variante, no a una copia del catálogo.
`Cliente.php`	Representa `clientes`, permite nombre/teléfono/dirección y email/`contrasena` para asignación. `pedidos()` devuelve la lista de pedidos del cliente. `$hidden` evita exponer `contrasena` en JSON. La migración llama al campo de contraseña `password`, mientras que el modelo usa `contrasena`; el flujo actual no utiliza ninguno de los dos para autenticación de clientes, así que no deben confundirse con el hash/login de administradores.
`Pedido.php`	Representa `pedidos`, con cliente, fecha, forma de entrega, estado, total y notas. `cliente()` es `belongsTo`; `detalles()` es `hasMany`. Desactiva los timestamps automáticos de Eloquent aunque la tabla tenga columnas `created_at` / `updated_at`; `store()` define la fecha de negocio explícitamente.
`PedidoDetalle.php`	Guarda pedido, variante, cantidad y precio unitario. `variante()` conecta con la opción del catálogo. El precio se guarda en la línea para conservar una referencia histórica aunque el catálogo luego cambie. Desactiva los timestamps automáticos aunque existan columnas correspondientes.
`Administrador.php`	El nombre singular y clave `id_administrador` se declaran de forma explícita. `HasApiTokens` habilita los tokens personales de Sanctum; `$fillable` permite solo nombre/correo/hash y `$hidden` oculta el hash al serializar.
`User.php`	Modelo de autenticación genérico creado por Laravel para tabla `users`, separado del administrador del negocio. Los atributos `Fillable` y `Hidden` controlan asignación/serialización; `HasFactory` y `Notifiable` agregan fábrica y notificaciones. `casts()` interpreta verificación como fecha y trata contraseña con cast `hashed`.

`$fillable` es una lista de autorización para asignación masiva: si un controlador pasa un array a `create()`, Eloquent solo acepta los campos declarados. No valida si el valor tiene el formato correcto; esa tarea corresponde a `$request->validate(...)`.

Migraciones: una historia versionada del esquema

Cada migración devuelve una clase anónima con `up()` (aplicar el cambio) y `down()` (revertirlo). Laravel registra cuál ya se ejecutó en la tabla `migrations`; editar una migración aplicada no vuelve a ejecutarla. Para un cambio nuevo se crea una migración posterior.

Infraestructura inicial de Laravel

- `0001_01_01_000000_create_users_table.php` crea `users` con nombre/correo único/contraseña/timestamps y campos de verificación; también crea `password_reset_tokens` y `sessions`. `down()` elimina esas tablas. El login de administrador de la tienda utiliza `administrador`, no esta tabla genérica.
- `0001_01_01_000001_create_cache_table.php` crea `cache` (clave, valor, vencimiento) y `cache_locks` (clave, dueño, vencimiento). La migración permite usar caché respaldada por DB; el tipo de caché se elige con configuración y en tests se sustituye por `array`.
- `0001_01_01_000002_create_jobs_table.php` crea trabajos pendientes, lotes y trabajos fallidos, con cola, intentos, payload, vencimiento y excepción. Es infraestructura del driver database; los endpoints actuales no ponen el registro del pedido en una cola.
- `2026_09_25_033012_create_personal_access_tokens_table.php` define el almacenamiento polimórfico que usa Sanctum: `tokenable` puede asociarse al administrador, se almacena el hash único y se guardan abilities/fechas de uso o expiración. El texto claro se entrega una sola vez al cliente, no se guarda en esta tabla.
- `2026_09_25_073424_create_administrador_table.php` crea la tabla singular propia, con ID, nombre, correo único y columna de contraseña. Su forma explica `$table` y `$primaryKey` en el modelo `Administrador`.

Tablas de catálogo y pedidos

- `2026_09_25_032643_create_productos_table.php` crea `productos`: `id_producto`, nombre, descripción opcional, precio decimal de dos lugares, categoría, imagen opcional y timestamps. Un precio decimal representa dinero con precisión de centésimos.
- `2026_09_25_032644_create_producto_variantes_table.php` crea ID de variante, FK a `productos.id_producto`, talle, color, stock y timestamps. La FK en cascada refleja que la variante pertenece al producto; el controlador aplica además protección del historial cuando edita.
- `2026_09_25_032645_create_clientes_table.php` crea el contacto y dirección, con email/contraseña opcionales heredados de un esquema más amplio. El flujo de compra solo requiere nombre, teléfono y texto de dirección/retiro.
- `2026_09_25_032646_create_pedidos_table.php` define ID, cliente por FK, fecha actual, tipo de entrega, estado con valor inicial Pendiente y total decimal. El estado se almacena como texto y el controlador limita sus transiciones permitidas.
- `2026_09_25_032647_create_pedido_detalles_table.php` define ID de detalle, FKs al pedido y variante, cantidad y precio unitario decimal. El precio unitario conserva lo acordado al crear la orden.
- `2026_10_01_031448_add_imagen_to_producto_variantes_table.php` añade `imagen` nullable a la variante después del stock. `down()` quita esa columna; permite que `ProductoController` priorice una foto específica de color/talle.
- `2026_11_02_122913_add_notas_to_pedidos_table.php` añade `notas` opcional para instrucciones del comprador y la elimina al revertir. `PedidoController` valida la longitud y la expone a administración.

Ajustes de datos ya sembrados

- `2026_09_29_180000_fix_seeded_product_images.php` actualiza imágenes de dos productos de ejemplo si coinciden nombre y URL anterior. `down()` los retorna a URLs externas anteriores. El doble filtro evita alterar indiscriminadamente cualquier otra imagen del catálogo.
- `2026_09_29_183000_use_classic_bra_asset.php` reemplaza la imagen de ejemplo del sostén por un recurso local, y el `down()` restituye el path previo. A diferencia de una migración de estructura, modifica filas existentes para mantenerlas coherentes con los assets.

Datos iniciales y fábrica

`database/seeders/DatabaseSeeder.php`

`run()` es el punto de entrada de `php artisan db:seed`; llama a `ProductoSeeder`. El trait `WithoutModelEvents` impide que eventos de modelo se disparen durante el seeding, y el import de `User` no cambia datos porque el archivo actual no utiliza esa clase. No se debe suponer que un seeder es idempotente: conviene inspeccionar qué tablas inserta antes de volver a ejecutarlo sobre datos que se quieran conservar.

`database/seeders/ProductoSeeder.php`

El array `$productos` es el catálogo de demostración: cada producto incluye datos comunes y una lista anidada de variantes. El bucle inserta primero producto para obtener su ID; luego `array_map()` añade ese ID y timestamps a cada variante y la inserta. Las rutas `/images/...` corresponden a recursos públicos del backend, no a las imágenes locales del bundle Vue.

Al final lee desde `config('admin...')` los datos bootstrap. Solo cuando correo y contraseña están presentes usa `updateOrInsert` para crear o actualizar el administrador; el nombre tiene un valor de respaldo. La contraseña se guarda con `bcrypt`. Esta vía de arranque es distinta del endpoint de registro protegido; los valores de entorno no se copian a este documento. Ejecuta este seeder en una base que entiendas: el catálogo de ejemplo inserta filas nuevas y no es una operación neutra.

`database/factories/UserFactory.php`

Laravel usa `definition()` para fabricar atributos de `User`: nombre/correo falsos, correo verificado, contraseña con hash y token de recordar sesión. `static::$password ??=` calcula el hash de ejemplo una vez por proceso y lo reutiliza al generar varias filas. `unverified()` devuelve otro estado de fábrica con `email_verified_at` nulo. La fábrica corresponde a `User`, no al administrador propio ni a productos; los tests del negocio construyen esos modelos directamente.

Pruebas: qué organizan y qué prueban

`tests/TestCase.php` extiende el caso base de Laravel y es el padre compartido de las pruebas. PHPUnit lee `phpunit.xml`, descubre los directorios Unit y Feature y define variables de prueba: SQLite en memoria, caché `array`, correo `array` y cola síncrona. `RefreshDatabase` en pruebas de características prepara el esquema aislado y evita escribir en la base de desarrollo.

`tests/Feature/OrderManagementTest.php`

- `test_customer_can_register_a_pending_order_using_server_prices` envía un total/precio falsos, espera total calculado con el catálogo, revisa precio histórico del detalle, notas y estado pendiente, y confirma que crear el pedido no baja stock.
- `test_order_creation_rejects_insufficient_stock_and_reused_captcha` prueba que una cantidad superior al stock no crea pedido ni cliente, y que un segundo uso del mismo CAPTCHA se rechaza.

- `test_admin_confirmation_deducts_stock_and_cancellation_restores_it_once` prueba listado administrativo, descuento al confirmar, que repetir confirmar no descuente otra vez, devolución al cancelar y rechazo de una transición inválida posterior.
- `test_admin_cannot_confirm_a_pending_order_after_stock_runs_out` crea dos pedidos que en conjunto sobrepasan stock; la confirmación del primero reduce existencia y la del segundo falla sin cambiar su estado pendiente.
- `test_order_management_requires_an_administrator_token` exige token en listado y actualización administrativa.
- `createVariant()`, `createOrder()`, `orderPayload()` y `adminToken()` son helpers de fixture: fabrican producto/variante, pedido válido con CAPTCHA almacenado en cache o token con ability `admin`. Centralizar datos evita repetir la preparación, no añade endpoints.

`tests/Feature/AdminManagementTest.php`

`RefreshDatabase` aísla los escenarios; `registrationData()` comparte los campos de alta. Los métodos, por tema, son:

- Registro/login y alta: `test_registration_requires_the_configured_admin_key`, `test_admin_can_register_login_and_create_a_product_with_variants` y `test_product_creation_requires_an_authenticated_admin`. Comprueban clave, hash, token y ruta protegida.
- Stock: `test_admin_can_update_variant_stock_to_zero` y `test_stock_update_requires_an_admin_and_rejects_negative_values`. Cero es válido; el número negativo no modifica la fila; sin token recibe 401.
- Imagen y datos: `test_admin_can_upload_a_product_image` usa `Storage::fake('public')` para probar upload sin escribir archivos reales; `test_product_creation_rejects_missing_database_fields` revisa validación antes de insertar.
- Edición: `test_admin_can_update_a_product_and_its_variants` comprueba conservar IDs existentes y crear nuevas; `test_product_update_rejects_variant_ids_from_another_product` impide cruzar productos; `test_product_update_rejects_duplicate_variant_ids` rechaza IDs repetidos; `test_product_update_preserves_variants_referenced_by_orders` protege variantes ya incluidas en `pedido_detalles`.
- Reemplazo/eliminación: `test_admin_can_replace_and_remove_a_product_image` comprueba archivos falsos, reemplazo de la ruta y limpieza de la imagen anterior/actual.
- `test_product_update_requires_an_authenticated_admin` comprueba que la edición también está protegida.

El test no se limita a mirar el status HTTP: varias pruebas verifican JSON, base de datos y filesystem falso. Esto cubre las consecuencias del flujo, por ejemplo que un `422` no haya cambiado variante o precio.

`tests/Feature/CaptchaTest.php`

- `test_correct_answer_is_accepted_only_once` pide el desafío por HTTP, deriva la suma visible, verifica que la primera solicitud dé `verified: true` y la segunda falle.
- `test_wrong_answer_and_honeypot_are_rejected` prueba una suma deliberadamente incorrecta y un campo trampa lleno.
- `test_challenge_endpoint_limits_requests_per_minute` excede diez solicitudes y espera `429` en la siguiente.

- `correctAnswer()` analiza el formato de pregunta y suma los dos enteros; es helper de prueba, no lógica duplicada en producción.

`tests/Feature/SecurityHardeningTest.php`

Comprueba que ya no están publicadas las antiguas rutas de prueba de pedido e imagen (`404` sin crear registros), que el catálogo responde lista, que el registro se cierra al existir un administrador y que un token con ability distinta de `admin` no alcanza las rutas de producto. Este último caso distingue "token válido" de "token autorizado".

`tests/Feature/ExampleTest.php` y `tests/Unit/ExampleTest.php`

Son ejemplos que vienen con Laravel: el primero pide `/` y espera `200`; el segundo ejecuta una aserción verdadera sin base de datos ni API. Sirven para mostrar la forma mínima de un test feature/unidad, pero no cubren la tienda.

Configuración Laravel, por archivo

Los archivos de `config/` convierten variables del entorno en valores que Laravel consulta con `config('...')`. En tests esos valores se pueden sobrescribir con `config([...])`; no hace falta imprimirlos ni conocer claves reales para entender su propósito.

Archivo	Qué contiene y quién lo consulta
`config/admin.php`	Puente a los nombres `ADMIN_REGISTRATION_KEY`, `ADMIN_BOOTSTRAP_NAME`, `ADMIN_BOOTSTRAP_EMAIL` y `ADMIN_BOOTSTRAP_PASSWORD`. `AdminAuthController` usa `registration_key`; `ProductoSeeder` usa los campos bootstrap. Solo se explican nombres, nunca valores.
`config/app.php`	Nombre, entorno, debug, URL base, zona horaria, idioma, locale de Faker, cifrado y modo de mantenimiento. `APP_KEY` es material sensible del entorno, no se documenta su contenido. `url()` usa la URL base para resolver recursos locales.
`config/auth.php`	Define guardia web por sesión y proveedor Eloquent para `User`, más parámetros estándar de restablecimiento/timeout. La API de administrador usa tokens Sanctum explícitos, no debe confundirse con ese guard web genérico.
`config/cache.php`	Selecciona store y configuración de array/database/file/Redis u otros drivers, claves y locks. `CaptchaController` requiere que el store soporte la vida del challenge; las pruebas eligen memoria `array`.
`config/cors.php`	Aplica a `api/*` y la cookie CSRF Sanctum. Lee orígenes permitidos de una variable separada por comas; métodos/cabeceras son configurables y `supports_credentials` está desactivado. CORS permite que un navegador lea respuestas, no autentica ni da permisos.
`config/database.php`	Lee la conexión por defecto y declara perfiles SQLite, MySQL/MariaDB, PostgreSQL y SQL Server; también fija tabla de historial de migraciones y opciones Redis. El test usa SQLite en memoria por `phpunit.xml`.
`config/filesystems.php`	Declara disco local privado, disco local `public` (raíz `storage/app/public`, URL `/storage`) y S3. `AdminProductoController` guarda imágenes en `public`; `storage:link` crea el enlace de lectura web.

Archivo	Qué contiene y quién lo consulta
`config/logging.php`	Canales stack/single/daily y opciones de stderr, syslog, errorlog, Slack/Papertrail y otros; el canal y nivel proceden del entorno. `ProductoController` manda excepciones al facade `Log`.
`config/mail.php`	Mailer por defecto y transportes SMTP, SES, Postmark, Resend, sendmail, log, array y fallover/roundrobin; define remitente. El flujo de tienda actual no manda email.
`config/queue.php`	Selecciona ejecución sync/database/Beanstalk/SQS/Redis u otras conexiones, política de reintento y tablas para lotes/fallos. Aunque hay tablas de queue, crear pedido se ejecuta sin despachar job.
`config/sanctum.php`	Dominios stateful, guard, caducidad opcional, prefijo y middleware de cookies/sesión. La respuesta de login crea su propio token con expiración; las rutas exigen habilidad mediante alias bootstrap.
`config/services.php`	Sección para credenciales de servicios externos (mail providers, AWS, Slack). No participan en catálogo/pedido hasta que el código invoque esos servicios.
`config/session.php`	Driver y vida de sesión, almacenamiento/cookie, encriptación, dominio, `http_only`, `same_site` y opciones de serialization. Se aplica a rutas web tradicionales; la SPA administrativa usa token bearer.

Un perfil disponible en configuración no implica que ese proveedor esté activo. Para saber el modo de ejecución, compara el archivo con la variable local sin mostrarla y con el código que llama al servicio.

Vista Laravel, estilos Vite y archivos de despliegue

- `resources/views/welcome.blade.php` es la vista de `/`. Su contenido largo representa la pantalla de bienvenida del esqueleto Laravel y carga los recursos Vite de Laravel; no contiene el storefront de Vue, que se desarrolla en el otro repositorio.
- `resources/css/app.css` es el punto de entrada CSS del esqueleto backend y contiene reglas mínimas; no es el `src/style.css` de la tienda.
- `resources/js/app.js` es el punto de entrada JS Laravel y actualmente está vacío. No arranca Vue.
- `vite.config.js` del backend compila assets propios de Laravel con los plugins Vite declarados allí; el otro repositorio tiene una configuración distinta para compilar `App.vue`.
- `package.json` del backend contiene dependencias/hook de frontend de Laravel y los comandos npm asociados; `composer.json` declara PHP, paquetes Laravel/Sanctum, autoload PSR-4 (`App\`, factories, seeders), y scripts (`setup`, `dev`, `test`, descubrimiento de paquetes). No copiar a mano dependencias del frontend Vue a este proyecto.
- `artisan` es el lanzador CLI. Incluye el autoloader de Composer, arranca la aplicación de consola y ejecuta el comando; por ejemplo `route:list`, `migrate` y `test` son comandos del framework, no scripts propios.
- `public/.htaccess` reescribe solicitudes de Apache para que la entrada de Laravel pueda procesarlas; `public/robots.txt` orienta rastreadores, no aplica control de acceso; `public/favicon.ico` es un icono binario.
- `.gitignore` excluye configuración privada, bases SQLite locales, resultados, logs y artefactos generados. `.editorconfig` recomienda convenciones de espacios; `.gitattributes` configura normalización/tratamiento de Git. `.npmrc` contiene opciones de npm del proyecto. Ninguno cambia reglas de negocio en ejecución.

- `AGENTS.md` y `CLAUDE.md` son instrucciones documentales para herramientas de desarrollo; `.claude/skills/**`, `boost.json` y `.mcp.json` configuran capacidades de agentes y no se cargan al recibir una petición HTTP. Se agrupan por función en vez de describir como código de tienda.
- `.env.example` sirve como plantilla de nombres y categorías de configuración, no como fuente para publicar valores. `.env` local se excluye por seguridad. Esta guía no transcribe valores del ejemplo, llaves o credenciales.

Recursos binarios y límites explícitos

`public/images/` contiene recursos fotográficos referenciados por seeders y respuestas del catálogo. Se documenta el papel de la carpeta, pero no se describe cada PNG/JPG como si fuese lógica: son archivos binarios y su inventario visual se valida por filename/ruta. Se excluyen igualmente `public/favicon.ico`, la base SQLite local, archivos de caché, resultados de PHPUnit, `vendor/`, `node_modules/`, lockfiles, logs, salidas de Vite/storage, PDFs y esta guía. Las plantillas de Laravel y las instrucciones de agentes se explican por función, pero no se consideran dependencias funcionales de la tienda.

Ejemplo final: cómo leer una regla completa de extremo a extremo

Al investigar "¿por qué la bolsa dice 4 unidades y el pedido recibe 422?", recorre los archivos en este orden:

1. `src/App.vue` forma una lista de `{id_variante, cantidad}` y envía el CAPTCHA al llamar al endpoint.
2. `src/services/api.js` transforma esa llamada en `POST /pedidos`.
3. `routes/api.php` la limita y la dirige a `PedidoController::store()`.
4. `bootstrap/app.php` convierte una excepción de validación API a JSON; el controlador verifica stock/precio desde la base, no desde la bolsa.
5. `ProductoVariante` enlaza stock e identidad con producto; la tabla de variantes contiene la cantidad persistida.
6. `submitOrder()` presenta los errores del `422` y actualiza catálogo si hace falta; no limpia bolsa ni abre el resumen WhatsApp ante fallo.
7. `OrderManagementTest` prueba tanto stock insuficiente como precio del servidor, y verifica que pedido/cliente no queden parcialmente creados.

Este método de lectura sigue una sola operación a través de frontera HTTP, enrutamiento, controlador, modelo, persistencia, respuesta y prueba. La misma técnica sirve para cambiar stock, guardar una imagen o proteger el acceso al panel.